



Full-stack software development

Front-end introduction: Next.js

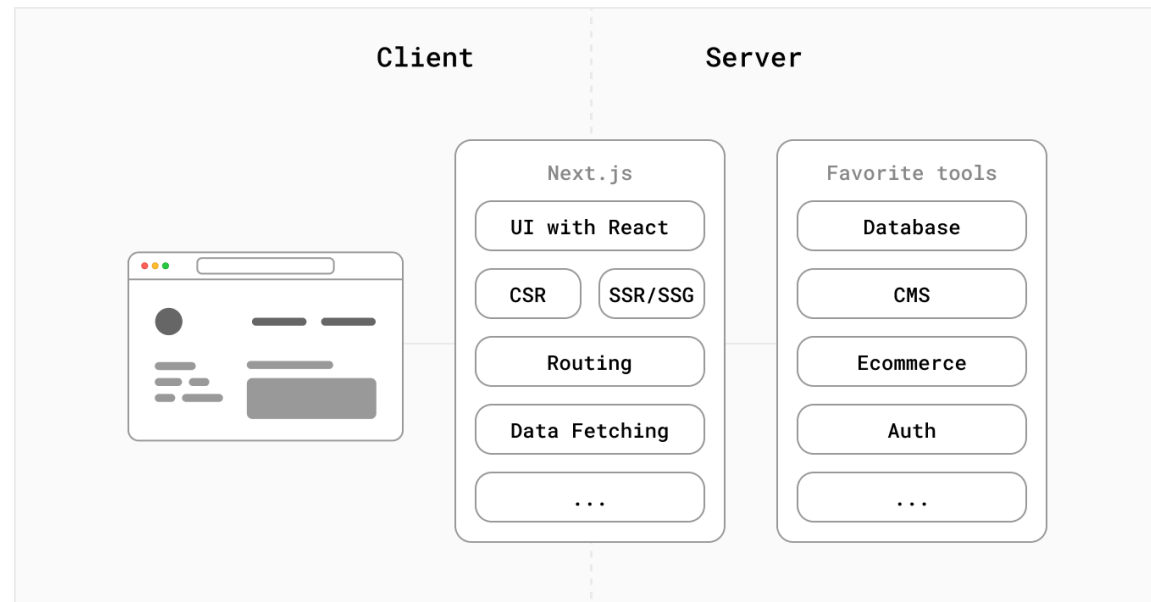
R. Naudts, J. Pieck, J. Rombouts, B. Van Impe



"Is a flexible React framework that gives you building blocks to create fast web applications"

Next.js

- A flexible **framework** built on top of React that gives you building blocks to create fast **web applications**
- Handles tooling and configuration needed for React and provides additional structure, features and optimizations.



Why Next.js?

- React is **unopinionated** about the other aspects of building applications
 - Building a complete React application from the ground up requires some effort.
 - Developers need to spend time configuring tools and reinventing solutions for common application requirements.

Next.js

Pages

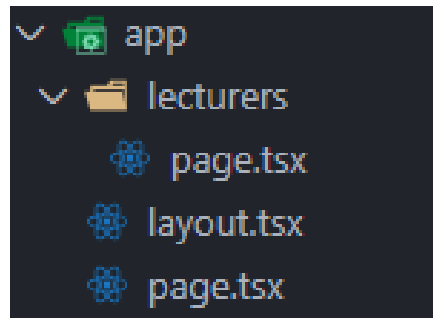
Routing

Handling static assets

CSS modules

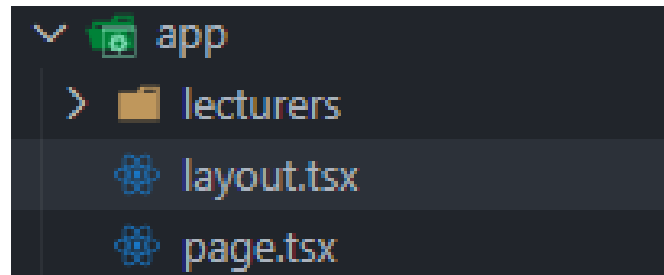
Pages

- A page is a React component which can be navigated to (= **routes**)
- Pages are associated with a route based on their file name
- For example
 - **app/page.ts** is associated with **/** route
 - **app/lecturers/page.ts** is associated with **/lecturers**



The Root Layout

- “*app/layout.tsx*” is the main template for your entire application. Every page is displayed inside this layout.
- This is where you define the `<html>` and `<body>` tags. It replaces the old “*_app.js*” from the Pages Router.



The Root Layout

```
import Header from '@components/header';
import '@styles/globals.css';

const RootLayout = async ({
  children,
  params,
}): {
  children: React.ReactNode;
  params: Promise<{ locale: string }>;
} => {
  const { locale } = await params;

  return (
    <html Lang={locale}>
      <body>
        <Header />
        <main className="p-6 min-h-screen flex flex-col items-center">
          {children}
        </main>
      </body>
    </html>
  );
};

export default RootLayout;
```

- The perfect place for elements that must appear on *every* page, such as a header, footer, or navigation bar.
- This is where you import global CSS files and wrap the application in “Context Providers” (e.g., for authentication).
- **{children}**: This prop, passed by Next.js, represents the content of the active page (page.tsx) that gets “inserted” into the layout.

Routing

- Use the **Link** component of **next/link** to link between pages in your application
- `<Link>` allows you to do client-side navigation and accepts props that give you better control over the navigation behaviour
- Client-side navigation: page transition happens using **JavaScript**, which is faster than the default navigation done by the browser

```
const Header: React.FC = () => {  
  return (  
    <header>  
      <a> Courses App</a>  
      <nav>  
        <Link href="/">Home</Link>  
        <Link href="/lecturers">Lecturers</Link>  
        <Link href="/schedule/overview">Schedules</Link>  
      </nav>  
    </header>  
  );  
};
```

Handling static assets

- There are two primary ways to handle static assets (e.g. images) in Next.js:
 - Public folder
 - Importing (recommended)

Handling static assets: public folder

- This folder is useful for assets that need a fixed URL and won't be processed, like favicon.ico.
- Files in the public directory are served from the root (/) of your application.
- You reference them with an absolute path starting with a slash.

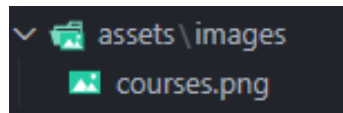


```

```

Handling static assets: importing

- Place your images in a folder within your project (e.g., assets/images) and import them directly into your .tsx file.
- Use the `<Image>` component from `next/image`.
- Next.js automatically optimizes images for size, format, and responsiveness.



```
import coursesLogo from '@assets/images/courses.png';  
  
<Image src={coursesLogo} alt="Courses Logo" />
```

CSS modules

- **Problem:** In large applications, accidentally reusing a class name like “.container” can cause style conflicts, where one component's styles unintentionally override another's.
- **Solution:** CSS Modules scope CSS locally to a specific component. This prevents styles from affecting other parts of your application.

CSS modules

- How it works:
 - Name your stylesheet with a .module.css extension. E.g. [home.module.css](#)
 - Write CSS in your css module.

```
.description {  
  text-align: center;  
  max-width: var(--max-width);  
  width: 100%;  
  z-index: 2;  
  font-family: var(--font-mono);  
}
```

- Import the module into your component. Next.js treats it as a JavaScript object.

```
import styles from '@styles/home.module.css';
```

- Use the imported styles object to assign className props.

```
<section className={styles.description}>
```